Flash Attention, Tile by Tile

From a plain attention block to PyTorch's flash kernels: padding lanes, stale shared memory,
and when `lse_sum != lse_sum` fires

SC26 nixnan tutorial

`flash-attn-expt`, branch `Flash-Attn`

October 2026

Roadmap

1. **A plain attention block**: $O = \mathrm{softmax}(\sigma QK^\top + \mathrm{mask})\,V$, and why it is memory-bound
2. **Massaging it into flash attention**: online softmax, tiles, one fused kernel; the split-KV variant and its combine kernel
3. **Padding lanes**: a decode step that fills one row of a 128×128 tile; the unloaded rows hold the *previous launch's* NaN-laden output
4. **How flash computes on them, and why discarding is safe**
5. **The combine guard `lse_sum != lse_sum`**: scenarios where it does *not* fire, and a proof that it cannot for any positive scale
6. **Making it fire**: a negative softmax scale

All numbers come from PyTorch 2.6's own flash source, rebuilt with dump hooks and verified **bitwise-equal** to PyTorch's kernel, running TinyGPT (4 layers, 4 heads, head dim 32) on an RTX 3090.

Attention in one line

For one head, with N queries, N_k keys and head dimension d:

$$S = \sigma\, QK^\top \in \mathbb{R}^{N \times N_k}, \qquad \sigma = \frac{1}{\sqrt{d}}$$

$$P_{ij} = \frac{e^{S_{ij}}}{\sum_{j'} e^{S_{ij'}}}, \qquad O = P\,V \in \mathbb{R}^{N \times d}$$

- Row i of O is a weighted average of the rows of V; the weights come from how well query i matches each key
- **Causal mask**: $S_{ij} = -\infty$ for $j > i$, so $P_{ij} = 0$
- **Rows are independent**: row i of O needs only row i of Q (and all of K, V)

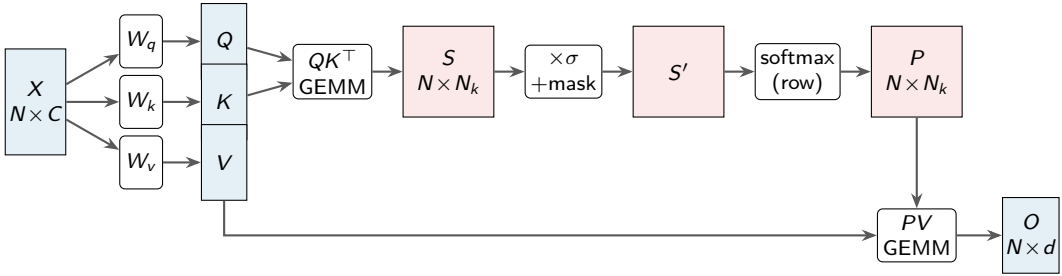
TinyGPT's attention

4 heads, $d = 32$, $\sigma = 1/\sqrt{32} \approx 0.177$
Q, K, V in FP16 (`q.half()` etc.)
`F.scaled_dot_product_attention` with the flash backend forced (as HF models such as BioMistral do)

Two kinds of calls

training / prefill: $N = N_k = T$, causal
decode (KV cache): $N = 1$ new query, $N_k =$ all keys so far, no mask needed

A plain attention block



- Written this way, every step is a separate kernel, and S, S' and P are $N \times N_k$ matrices written to and read from GPU memory (HBM)
- For $N = N_k = 4096$, one head: 16M entries per matrix, versus 128K for Q ($d = 32$)
- The arithmetic is cheap; **the traffic dominates**. Attention is memory-bound and quadratic in memory

Softmax in practice: max subtraction and the log-sum-exp

- e^S overflows FP32 for $S > 88.7$, so every implementation subtracts the row max $m_i = \max_j S_{ij}$:

$$P_{ij} = \frac{e^{S_{ij}-m_i}}{\ell_i}, \qquad \ell_i = \sum_j e^{S_{ij}-m_i}$$

- Every exponent is ≤ 0; the maximal term is exactly $e^0 = 1$, so $\ell_i \in [1, N_k]$
- The **log-sum-exp** summarizes a row:

$$\mathrm{lse}_i = \log \sum_j e^{S_{ij}} = m_i + \log \ell_i$$

- With it, $P_{ij} = e^{S_{ij}-\mathrm{lse}_i}$. Flash stores it for the backward pass and to merge partial results

Two edge cases a kernel must handle

fully masked row: every $S_{ij} = -\infty$, $m_i = -\infty$, $e^{-\infty-(-\infty)} = \mathrm{NaN}$. Kernels special-case $m = -\infty$.

NaN anywhere in the row: $\ell_i = \mathrm{NaN}$.

Flash's `softmax.h` guards both: if $\ell = 0$ or $\ell \neq \ell$, it sets the LSE to $\pm\infty$ instead of $m + \log \ell$.

The flash idea: never materialize S and P

1. **Tile**: cut Q into row blocks Q_i and K, V into row blocks K_j, V_j
2. **Fuse**: one kernel does $QK^\top$, scale, mask, softmax and PV for a tile, entirely in registers and shared memory
3. **Online softmax**: process the key blocks one at a time, keeping per row only a running max m, a running sum ℓ and an *unnormalized* output accumulator $\tilde{O}$

Plain

HBM traffic $\Theta(NN_k)$ for S, P
5+ kernels

Flash

HBM traffic $\Theta((N + N_k)\, d)$: read Q, K, V once per tile row, write O and the LSE
1 kernel (plus a small combine kernel if KV is split)

The price: a tile is a fixed size (128×128 here). **When the sequence does not fill it, some lanes compute on data that is not part of the problem.** That is the subject of part 3.

Online softmax: merging key blocks one at a time

For one row, after key blocks $1..j-1$ we hold m, ℓ, $\tilde{O}$. For block j with scores s:

$$m' = \max(m, \max s)$$
$$\ell' = e^{m-m'}\,\ell + \sum e^{s-m'}$$
$$\tilde{O}' = e^{m-m'}\,\tilde{O} + e^{s-m'}\,V_j$$

After the last block: $O = \tilde{O}/\ell$, $\mathrm{lse} = m + \log \ell$.

The factor $e^{m-m'} \le 1$ rescales what was accumulated under the old max. The result is exact, not an approximation.

Worked example: scores [1, 3 | 2, 0]

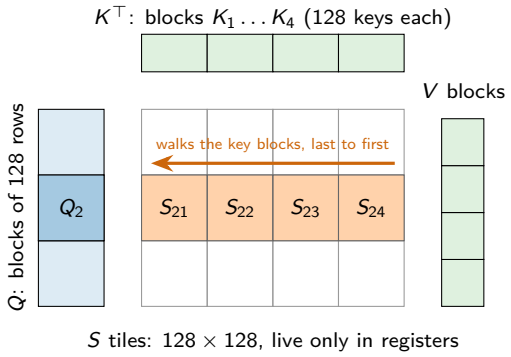
block 1: $m = 3$, $\ell = e^{-2} + e^{0} = 1.135$

block 2: $m' = \max(3, 2) = 3$,
$\ell' = e^{0} \cdot 1.135 + e^{-1} + e^{-3}$
$= 1.135 + 0.368 + 0.050 = 1.553$

direct: $e^{-2} + 1 + e^{-1} + e^{-3} = 1.553$ ✓

$\mathrm{lse} = 3 + \log 1.553 = 3.440$

Tiling: one thread block (CTA) per query block



- PyTorch 2.6, FP16, $d = 32$:
  `Flash_fwd_kernel_traits<32,128,128,4>`
  - tile $k_M \times k_N = 128 \times 128$ (queries $\times$ keys)
  - 4 warps $= 128$ threads per CTA
  - grid $(\lceil N/128 \rceil,$ batch, heads$)$
- CTA i keeps Q_i in **shared memory**, streams K_j, V_j through shared memory, and keeps S_{ij}, P_{ij}, $\tilde{O}_i$, m, ℓ in **registers**
- $QK^\top$ and PV run on tensor cores: `HMMA.16816.F32` (FP16 in, FP32 accumulate)

The flash kernel, step by step (`flash_fwd_kernel.h`)

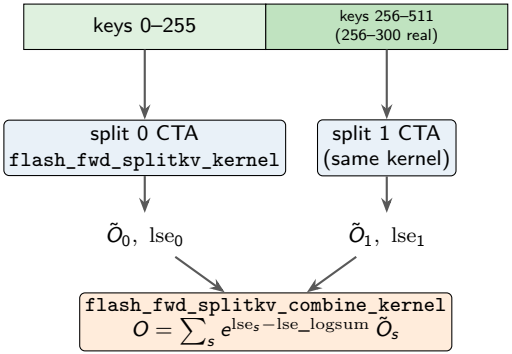
```
load Q_i -> sQ            // rows < seqlen_q only
for n_block = last .. first:
  load K_j -> sK, V_j -> sV  // V: OOB rows = 0
  acc_s = Q_i K_j^T       // HMMA, FP32 regs
  mask(acc_s)             // OOB / causal: = -inf
  softmax_rescale_o(acc_s, acc_o)
    // m' = max(m, rowmax)
    // P = 2^(S*s2 - m'*s2), update l
    // acc_o *= 2^((m - m')*s2)
  acc_o += P V_j          // HMMA
lse = normalize_softmax_lse(acc_o) // guard
sO = FP16(acc_o), staged AT THE ADDRESS OF sQ
copy sO -> O,  rows < seqlen_q only
store lse,     rows < seqlen_q only
```

- Base 2: $\sigma_2 = \sigma \log_2 e$, so $e^{\sigma(x-m)} = 2^{x\sigma_2 - m\sigma_2}$, one `MUFU.EX2`
- Masking *assigns* $-\infty$; it does not add
- The output staging reuses the Q buffer: Q_i is no longer needed
- Stores are **predicated on the row index**

The per-row guard (`softmax.h:174-177`)

```
sum = row_sum(mi);
lse = (sum == 0.f || sum != sum)
  ?  (Split ?  -INFINITY : INFINITY)
  :  row_max * softmax_scale + __logf(sum);
```

Split-KV: when there are too few query blocks



- Decode: $N = 1$ gives 1 query block per head, too few CTAs for 82 SMs
- `num_splits_heuristic` splits the keys; split kernel tile 64×256, so splitting needs $N_k > 256$
- Each split writes a partial output and its own LSE. **`Split=true`**: a zero or NaN row sum gives $\mathrm{lse}_s = -\infty$
- A split with no key block (causal) writes $-\infty$ directly
- The combine kernel merges the splits per row, 16 rows per block

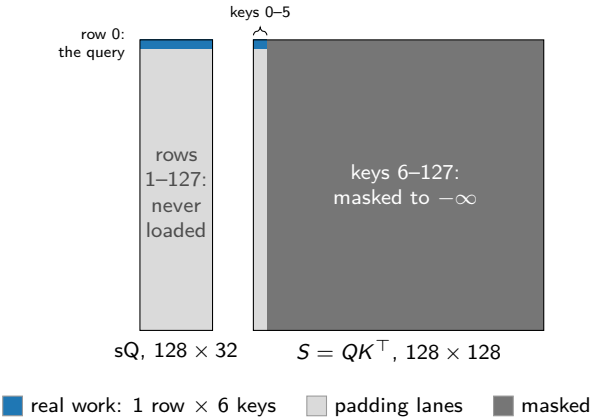
The combine kernel and its guard

```
lse_max = lse_max == -INFINITY ? 0.0f : lse_max;  // In case all local LSEs are -inf
float lse_sum = expf(lse_accum(0) - lse_max);
for (int l = 1; l < kNLsePerThread; ++l) { lse_sum += expf(lse_accum(l) - lse_max); }
lse_sum = Allreduce<kRowsPerLoadTranspose>::run(lse_sum, sum_op);
// For the case where all local lse == -INFINITY, we want to set lse_logsum to INFINITY. Otherwise
// lse_logsum is log(0.0) = -INFINITY and we get NaN when we do lse_accum(l) - lse_logsum.
ElementAccum lse_logsum = (lse_sum == 0.f || lse_sum != lse_sum) ? INFINITY : logf(lse_sum) +
    lse_max;
```

- $\mathrm{lse_logsum} = \log \sum_s e^{\mathrm{lse}_s}$, computed with max subtraction
- scales $e^{\mathrm{lse}_s - \mathrm{lse_logsum}}$ weight the partial outputs

- `lse_sum == 0`: all splits $-\infty$. Writing $+\infty$ makes the scales $e^{-\infty-\infty} = 0$, not $e^{-\infty+\infty} = \mathrm{NaN}$
- **`lse_sum != lse_sum`**: the comment gives no case. Part 5 shows why

A decode step in a 128×128 tile



TinyGPT completes 312; the last forward pass is a decode step per layer:

- $N = 1$ query, $N_k = 6$ keys (BOS, 3, 1, 2, and two generated digits)
- one CTA per head, one 128×128 tile
- **1 of 128 query rows and 6 of 128 key columns are real**: 24 of 16,384 scores per head (0.15%)

The other 99.85% are *padding lanes*. The tensor cores compute them anyway: an MMA instruction covers the whole fragment.

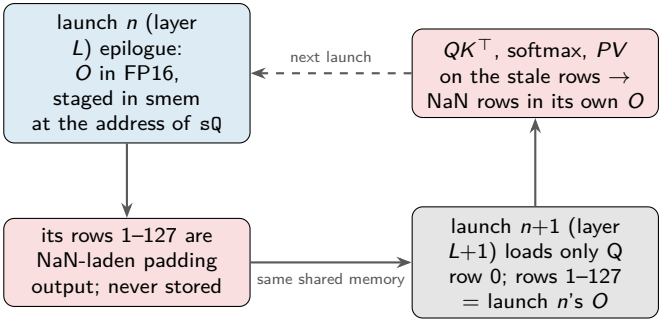
What flash loads, and what it deliberately does not

```
// We don't need to clear the sQ smem tiles since we'll only write out the valid outputs
copy<Is_even_MN, Is_even_K>(gmem_tiled_copy_QKV, tQgQ, tQsQ, tQcQ, tQpQ, actual_seqlen_q - m_block *
    kBlockM);
...
// We don't need to clear the sK smem tiles since we'll mask out the scores anyway.
copy<Is_even_MN, Is_even_K>(gmem_tiled_copy_QKV, tKgK, tKsK, tKVcKV, tKVpKV, actual_seqlen_k -
    n_block * kBlockN);
...
copy<Is_even_MN, Is_even_K, /*Clear_OOB_MN=*/true>(gmem_tiled_copy_QKV, tVgV, tVsV, ...);
```

tile	rows beyond the sequence	why it is safe (claimed)	cost saved
sQ	**not loaded, not cleared**	those output rows are never stored	a clear per launch
sK	**not loaded, not cleared**	those scores are masked to $-\infty$	a clear per key block
sV	cleared to 0	$P = 0$ there, but $0 \cdot NaN = NaN$, so V *must* be clean	—

So the padding rows of sQ and sK hold **whatever the shared memory last contained**. What is that?

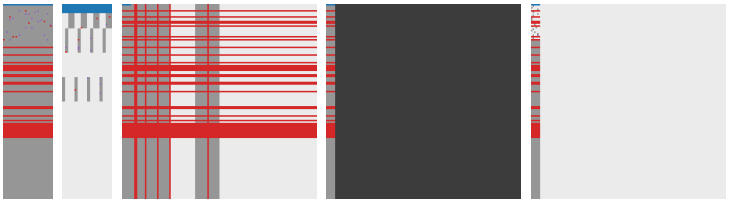
Scenario: the last launch's NaN-laden output becomes this launch's Q



Measured, calls 8–11 (the 4 layers of the last decode step), 508 stale Q rows each:

- **416** stale Q rows are bit-identical to rows of an earlier call's O tile
- call 8's 130 NaN O rows → call 9's stale Q has 3,234 NaN values
- the other stale rows: zero, or from other kernels on the same SM
- stale K rows: 404 zero, 84 old K rows
- a second identical run: **different** stale tiles, **bitwise-identical** output

The dumped tiles (call 11 = layer 3, head 0, every element)



- rows top to bottom = queries; columns = head dim (sQ, sK) or keys
- the real work: the blue segment at the top left
- NaN rows of S: stale Q rows with NaN
- NaN columns of S: stale K rows with NaN
- the mask overwrites the columns; **the rows survive into** P

How flash computes on the stale lanes (call 11, 4 heads)

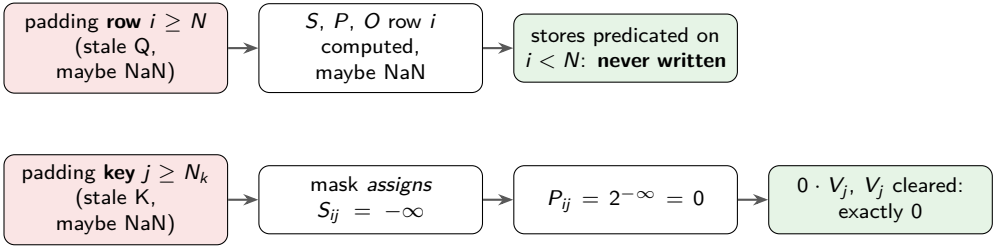
region	values	NaN
Q row 0 (loaded)	128	0
Q rows 1–127 (stale)	16,256	**3,239**
K rows 6–127 (stale)	15,616	13
V rows 6–127 (cleared)	15,616	0 (all 0)
S, real row, keys 0–5	24	0
S, real row, padding keys	488	13
S, padding rows	65,024	**18,274**
S masked, padding rows	65,024	798 ($+61{,}976$ $-\infty$)
P, padding rows	65,024	**798**
O, padding rows	16,256	**4,256**
O, row 0 (the output)	128	**0**

Follow one stale row with a NaN in its Q:

1. S row $= q \cdot K^\top$: NaN in every column
2. mask: columns 6–127 $\to -\infty$; NaN stays in columns 0–5
3. row max ignores NaN (`fmaxf`): $m = -\infty$, so `max_scaled`$= 0$
4. $P = 2^{NaN}$: NaN in 6 columns, 0 elsewhere
5. $\ell =$ NaN $\Rightarrow$ `softmax.h`: `sum != sum`, $\mathrm{lse} = +\infty$
6. O row $= PV$: NaN in all 32 columns

133 such rows: $133 \times 6 =$ **798** NaN in P,
$133 \times 32 =$ **4,256** in O

Why the results are discarded, and why that is safe



- **Rows are independent** (slide 3): a padding row's garbage cannot reach a real row, so it can be computed and dropped
- **Keys are mixed into every row**, so padding keys must be neutralized: the mask overwrites the score, and V is cleared so that $0 \cdot V = 0$
- **Why not clear?** Clearing sQ and sK costs shared-memory stores every launch, for lanes that never matter. Predicated stores are free
- Also dropped: the padding rows' LSE ($+\infty$ from the guard)
- The real row's 13 NaN padding-key scores are masked to $-\infty$: harmless

What this means for a binary-level NaN detector

- The NaNs are **real values in real registers**: `HMMA` operands, `FADD`/`FMUL` in the softmax, `MUFU.EX2`
- A tool that checks instruction operands (nixnan) reports them: **thousands of NaN sites** in `flash_fwd_kernel` while every tensor the model sees is finite
- Counts vary from run to run and irregularly with sequence length: they depend on what the previous kernel on that SM left in shared memory
- `softmax.h`'s `sum != sum` disjunct fires on padding rows (133 of 508 in call 11), never on real rows
- The same pattern appears in any tiled kernel: cuBLAS/CUTLASS GEMMs, PyTorch's warp softmax (which pads with $-\infty$)

Lesson

A NaN in a register is not a NaN in the program. To tell them apart you need the lane's row and column against the sequence bounds, or you check what is stored.

Scenarios A and B: the combine guard on clean and poisoned input

301-key prompt, so PyTorch picks 2 KV splits for every call; 12 calls, 4 heads. B adds $+\infty$ to one coordinate of the last prompt embedding.

	A (clean)	B ($+\infty$)
split `softmax.h`, real rows: `sum != sum` $\rightarrow -\infty$	0	**1,144**
split `softmax.h`, padding rows: `sum != sum`	1,588	3,931
combine guard fired, real rows	0	**568**
via `lse_sum == 0`	0	568
via `lse_sum != lse_sum`	**0**	**0**
combine guard, padding rows (16-row block tail), `== 0`	144	144
output rows with NaN	0	752

- Even with an ∞ injected, **the NaN disjunct never fires**
- The NaN is removed one level earlier: in the split kernel, `softmax.h` maps a NaN row sum to $\mathrm{lse}_s = -\infty$
- The combine then sees only finite or $-\infty$ LSEs; when all are $-\infty$, $\mathrm{lse_sum} = 0$ exactly
- The guard cleans the LSE, **not** O: 752 NaN rows vs 568 firings ($0 \cdot \mathit{NaN}$ in $O \mathrel{+}= e^{\cdots} \tilde{O}_s$)

Why it cannot fire: the argument in three steps

1. **Combine.** $\mathrm{lse_sum} = \sum_s e^{\mathrm{lse}_s - L}$, $L = \max_s \mathrm{lse}_s$. It is NaN only if some lse_s is NaN, or some $\mathrm{lse}_s = +\infty$ ($L = +\infty$, $\infty - \infty$).
   Otherwise $\mathrm{lse_sum} \in \{0\} \cup [1, n_{\text{splits}}]$.
2. **Split kernel.** $\mathrm{lse}_s = (\ell = 0 \vee \ell \neq \ell)\ ?\ -\infty : m\sigma + \log \ell$.
   So $\mathrm{lse}_s \in \{NaN, +\infty\}$ needs $\ell = +\infty$ (or $m\sigma = +\infty$ with finite ℓ).
3. **Softmax terms.** Each term is $2^{\mathrm{fl}(x\sigma_2) - \mathrm{fl}(m\sigma_2)}$, and $x \leq m$. For $\sigma > 0$:
   - $\mathrm{fl}(m\sigma_2)$ finite: every exponent ≤ 0, every term in $[0, 1]$, the max term is 1, so $\ell \in [1, N_k]$, finite
   - $\mathrm{fl}(m\sigma_2) = \pm\infty$ (overflow or $m = +\infty$): the max term is $2^{\infty - \infty} = \mathrm{NaN}$, so $\ell = \mathrm{NaN} \to$ guard $\to -\infty$
   - $m = -\infty$ (all masked): `max_scaled`$= 0$, every term $2^{-\infty} = 0$, $\ell = 0 \to -\infty$

Proposition

For any $\sigma > 0$ and *any* FP16 Q, K, V (including $\pm\infty$ and NaN), every $\mathrm{lse}_s \in \mathbb{R} \cup \{-\infty\}$, so `lse_sum != lse_sum` is false. TinyGPT ($1/\sqrt{32}$) and BioMistral ($1/\sqrt{128}$) can never take it.

A codegen detail the proof depends on, and huge positive scales

Source (`softmax.h`):

```
// max * log_2(e)) This allows the compiler to use
    the
// ffma instruction instead of fadd and fmul
    separately.
tensor(mi, ni) = exp2f(tensor(mi, ni) * scale -
    max_scaled);
```

Compiled split kernel (SASS): **not fused**

```
FMUL R28, R28, c[0x0][0x238] ;      // x * scale
...
FADD R154, -R47, R154 ;             // - max_scaled
FSETP.GEU.AND P1, PT, R160, -126, PT ;
MUFU.EX2 R160, R160 ;
```

Two roundings, so overflow of $m\sigma_2$ yields $\infty - \infty$ = NaN, which the guard catches.

Raw flash call, $N_k = 301$, 2 splits, head 0 (our build and PyTorch's agree, NaN-aware):

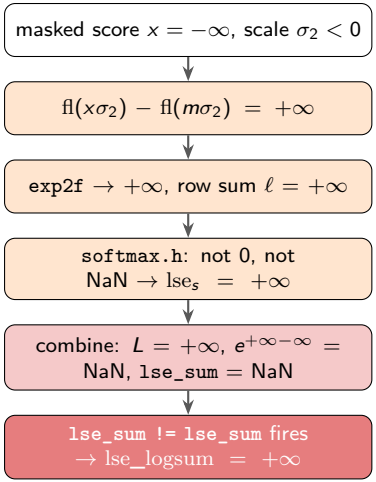
σ	split 0: m, ℓ, lse	combine	fires
$1/\sqrt{32}$	15.2, 33.1, 6.19	1.23	0/4
10^{30}	-37.1, 256, $-3.7 \cdot 10^{31}$	1	0/4
10^{37}	-37.1, **NaN**, $-\infty$	1	0/4

At $\sigma = 10^{37}$, $m\sigma_2$ overflows; unfused code gives ℓ = NaN and `softmax.h` catches it.

A *fused* `FFMA` would give $\ell = +\infty$ and open the door, but only for $\sigma > 1.7 \cdot 10^{27}$ with FP16 inputs.

A negative softmax scale breaks the premise

- The proof used $x \le m \Rightarrow x\sigma_2 \le m\sigma_2$.
  With $\sigma < 0$ the order flips: the exponent is $(m - x)\,|\sigma_2| \;\ge 0$
- A **masked** score $x = -\infty$:
  $(-\infty) \cdot \sigma_2 = +\infty$, so the term is $2^{+\infty} = +\infty$
- Large score spreads overflow too:
  $(m - x)|\sigma_2| > 128$
- $\ell = +\infty$ is neither 0 nor NaN: `softmax.h` passes it, $\mathrm{lse}_s = m\sigma + \log\infty = +\infty$
- `F.scaled_dot_product_attention(..., scale=-0.1)` is accepted: neither SDPA nor `mha_fwd` checks the sign



It fires: one raw flash call (`trigger_nan_guard.py`)

Decode-shaped call: $N = 1$, $N_k = 301$, 2 splits, 4 heads, random FP16 Q, K, V. Head 0:

σ	split 0: m, ℓ, lse	split 1: m, ℓ, lse	`lse_sum`	disjunct
$1/\sqrt{32}$	15.23, 33.1, 6.19	17.08, 5.52, 4.73	1.23	none, 0/4
10^{37}	-37.1, NaN, $-\infty$	17.08, 1, $1.7 \cdot 10^{38}$	1	none, 0/4
-0.1	15.23, 1435, 5.75	17.08, $+\infty$, $+\infty$	**NaN**	**!=, 4/4**
-1	15.23, $1.9 \cdot 10^{16}$, 22.3	17.08, $+\infty$, $+\infty$	NaN	!=, 4/4
-10	15.23, $+\infty$, $+\infty$	17.08, $+\infty$, $+\infty$	NaN	!=, 4/4

- At $\sigma = -0.1$, split 0 is still finite (all 256 keys real, spread $< 128/|\sigma_2|$)
- Split 1 covers keys 256–511, of which 301–511 are beyond the sequence: **211 masked columns**, each $2^{+\infty}$
- So the trigger is the **mask on the padding keys**, the same lanes as part 3
- Every case matches PyTorch's own kernel

It fires inside TinyGPT: `dump_lse.py -input A -scale -0.1`

Clean input A, every attention call with $\sigma = -0.1$:

calls	fired	via == 0	via !=
0: layer 0 prefill (301 rows/head)	1,200	0	**1,200**
1–3: layers 1–3 prefill	3,612	3,612	0
4, 8: layer 0 decode	8	0	**8**
5–7, 9–11: layers 1–3 decode	24	24	0
total	4,844	3,636	**1,208**
default scale	0	0	0

Completion: 312452 becomes . . . 312000.

- **Layer 0**: clean inputs; the causal and padding masks supply the $-\infty$ scores $\Rightarrow \ell = +\infty$, $\mathrm{lse}_s = +\infty$, `lse_sum != lse_sum`
- Only row 255 of each head escapes: it sees all of split 0 unmasked, and split 1 has no key block for it
- **Layers 1–3**: inputs already NaN from layer 0, so `softmax.h` maps every split to $-\infty$, and the guard fires via `lse_sum == 0`, as in scenario B

What a negative scale would mean in practice

- $\mathrm{softmax}(-|\sigma|\, QK^\top)$ attends most to the keys *least* similar to the query: "anti-attention". No standard model does this; every trained transformer uses $\sigma = 1/\sqrt{d} > 0$
- Realistic ways to get there: a sign error, or a user-supplied `scale=` computed wrongly. The API accepts it silently
- It is an **engineered trigger**: the cheapest input that reaches the disjunct, not a scenario BioMistral or TinyGPT can hit in normal use
- Even when the guard fires, it only fixes the LSE: the outputs of the affected rows are non-finite (1,204 rows in layer 0) and the completion degrades

So what is `lse_sum != lse_sum` for?

Defensive code. With unfused codegen and $\sigma > 0$ it is dead. It becomes live with $\sigma < 0$, or with a fused-`FFMA` build and an absurdly large scale (or BF16 inputs and $\sigma > 0.69$).

Summary

1. **Flash attention** = tiled, fused attention with an online softmax: no $N \times N$ matrix ever reaches memory; per row it keeps m, ℓ, $\tilde{O}$ and emits O and $\mathrm{lse} = m\sigma + \log \ell$
2. **Fixed tiles create padding lanes.** A decode step uses 24 of 16,384 scores per head
3. **Unloaded rows hold stale shared memory**, typically the previous launch's NaN-laden output tile, because O is staged at the Q buffer's address
4. **Flash computes on them and discards them**: padding rows are never stored (rows are independent); padding keys are overwritten with $-\infty$ and V is cleared ($0 \cdot V = 0$)
5. `lse_sum != lse_sum` **never fires for** $\sigma > 0$, for any input: `softmax.h` turns every NaN row sum into $-\infty$ first, and ℓ cannot reach $+\infty$
6. **A negative scale makes it fire**: masked scores become $2^{+\infty}$, $\mathrm{lse}_s = +\infty$, $\infty - \infty$ in the combine. 1,208 firings in TinyGPT

Code and traces: `flash-attn-expt/PyTorch_Instrumentation/`. Full report: `flash-attn-expt/latex/report.pdf`.